17

BÖLÜM: KLASIK SENKRONIZASYON, MONITÖRLER & KILITLER | MODULE: CLASSICAL SYNCHRONIZATION, MONITORS & LOCKS

Condition Değişkenleri: Çoklu Bekleme Kuyrukları (notFull & notEmpty)

Condition Variables: Multi-Queue Coordination (notFull & notEmpty)

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────┐
│              TEK BİR REENTRANTLOCK İÇİNDE ÇİFT BEKLEME KUYRUĞU              │
└──────────────────────────────────────────────────────────┘

ReentrantLock lock = new ReentrantLock();
Condition notFull  = lock.newCondition(); // Üreticilerin bekleme kuyruğu
Condition notEmpty = lock.newCondition(); // Tüketicilerin bekleme kuyruğu

[ ÜRETİCİ (PUT) ]:
Kuyruk doluysa ──► notFull.await() ──► Sadece notFull kuyruğunda bekler!
Eleman ekleyince ──► notEmpty.signal() ──► YALNIZCA Tüketicileri uyandırır!

[ TÜKETİCİ (TAKE) ]:
Kuyruk boşsa ──► notEmpty.await() ──► Sadece notEmpty kuyruğunda bekler!
Eleman çekince ──► notFull.signal() ──► YALNIZCA Üreticileri uyandırır!

AVANTAJ: synchronized gibi herkesi tek bir torbaya atıp uyandırmaz (Sıfır İtişme)!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski sistemde (synchronized wait/notify) lokantada hem yemek bekleyen aç müşteriler hem de boş tabak bekleyen bulaşıkçılar aynı odada bekliyordu. Aşçı 'Yemek hazır' dediğinde herkesi aynı anda uyandırıyordu (notifyAll). Bulaşıkçılar uyanıp 'Bize yemek lazım değil ki' diyip tekrar uyuyordu (Boşa enerji). Condition ise lokantaya iki ayrı kapı koymaktır: 1. Kapı: 'Yemek Bekleyenler' (notEmpty). 2. Kapı: 'Temiz Tabak Bekleyenler' (notFull). Aşçı yemek yapınca sadece müşterilerin kapısını çalar; bulaşıkçıları boş yere uyandırmaz!

🇬🇧 ENGLISH (ELI5)

In intrinsic `wait/notify`, producers and consumers share a single noisy waiting room. When a producer delivers an item and calls `notifyAll()`, both idle producers and consumers wake up, frantically thrashed from sleep only to realize the notification wasn't for them. `Condition` splits the room into targeted queues: `notFull` for producers and `notEmpty` for consumers. Producers wake only consumers; consumers wake only producers. Zero wasted CPU cycles!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Condition Mimarisi ve ArrayBlockingQueue: 1) Çoklu Bekleme Seti: Tek bir kilit üzerinden `lock.newCondition()` ile onlarca farklı koşul kuyruğu oluşturulabilir. 2) await() ve signal(): `await()` kilidi bırakıp thread'i o Condition'a özel AQS bekleme kuyruğuna sokar. `signal()` ise sadece o kuyruğun başındaki tek bir thread'i kilit için yarışmaya gönderir. 3) Neden signal() güvenlidir?: Çünkü `notEmpty.signal()` çağrıldığında uyandırılan kişinin KESİNLİKLE bir tüketici olduğu garanti altındadır! `notify()` gibi yanlışlıkla başka bir üreticiyi uyandırıp sistemi kilitleme tehlikesi yoktur. JDK'nın `ArrayBlockingQueue` sınıfı tamamen bu desenle yazılmıştır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Condition Queues and ArrayBlockingQueue Topology: 1) Dedicated AQS Wait Nodes: Each `ConditionObject` instance maintains its own first/last FIFO node pointers. When a thread executes `await()`, AQS saves the hold count, fully releases the lock, and enqueues the thread in the condition queue. 2) Targeted Signaling: Calling `signal()` unlinks the first node from the condition queue and transfers it to the AQS sync queue (`_EntryList` equivalent). 3) Safety of `signal()` over `signalAll()`: Because distinct conditions cleanly isolate producers from consumers, calling `notEmpty.signal()` is 100% immune to the missed-signal hazards that plague intrinsic `notify()`. This is the exact implementation powering `java.util.concurrent.ArrayBlockingQueue`.

⚡ Modern Java 25 LTS Kodu ⚡ Modern Java 25 Implementation

```
// Java 25: Condition Değişkenleri ile Yüksek Başarımlı Bounded Queue
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

public class ConditionQueue<T> {
    private final Object[] items;
    private int count, putIndex, takeIndex;

    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notFull  = lock.newCondition(); // Üretici kuyruğu
    private final Condition notEmpty = lock.newCondition(); // Tüketici kuyruğu

    public ConditionQueue(int capacity) {
        this.items = new Object[capacity];
    }

    public void put(T x) throws InterruptedException {
        lock.lock();
        try {
            while (count == items.length) {
                notFull.await(); // Yer yoksa üretici bekler
            }
            items[putIndex] = x;
            if (++putIndex == items.length) putIndex = 0;
            count++;
            notEmpty.signal(); // Sadece tüketicilere sinyal ver!
        } finally {
            lock.unlock();
        }
    }

    @SuppressWarnings("unchecked")
    public T take() throws InterruptedException {
        lock.lock();
        try {
            while (count == 0) {
                notEmpty.await(); // Veri yoksa tüketici bekler
            }
            T x = (T) items[takeIndex];
            if (++takeIndex == items.length) takeIndex = 0;
            count--;
            notFull.signal(); // Sadece üreticilere sinyal ver!
            return x;
        } finally {
            lock.unlock();
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı ⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: 'Condition.await() çağrılmadan önce lock.lock() çağrılmazsa ne olur?' Yanıt: 'Tıpkı synchronized olmadan wait() çağrıldığında olduğu gibi IllegalMonitorStateException fırlatılır! Condition'lar ait oldukları ReentrantLock'a göbekten bağlıdır.'

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Critical Requirement: Invoking `condition.await()` or `condition.signal()` without holding the associated `ReentrantLock` immediately throws `IllegalMonitorStateException`. The calling thread must hold the lock that owns the condition.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Condition, tek bir kilit altında birden fazla bağımsız bekleme kuyruğu oluşturur.

• notFull ve notEmpty ayrımı sayesinde signal() güvenle kullanılabilir.

• JDK'nın ArrayBlockingQueue sınıfı Condition mimarisi üzerine inşa edilmiştir.

🎯 Key Takeaways for the Interview

• Condition variables decouple wait-sets into distinct semantic queues.

• Condition queues make signal() completely safe by targeting specific thread roles.

• ArrayBlockingQueue is the canonical enterprise implementation of Condition queues.